Bayesian Visualization Explanations Research Progress Documentation:

Name: Alessandra Priolo

PI: Dr. Berger

Department: Department of Computer Science

Number of Research Credits: 1 credit

Semester: Spring 2023

Table of Contents:

1. **Introduction**
2. **Method**
3. **Code Documentation**
    a. Priolo_laplace_approx.py script
    b. train_places.py
    c. Vector_to_params_priolo.py
    d. make_predictions_priolo.py
4. **Documentation of Progress**
    a. 01/06/2023-02/09/2023
    b. 02/15/2023
    c. 02/22/2023
    d. 03/01/2023
    e. 03/08/2023
    f. 03/22/2023
    g. 04/05/2023
    h. 04/12/2023
    i. 04/19/2023
    j. 04/26/2023
    k. 05/02/2023

1. Introduction:

The work documentation outlines the process of training a ResNet18 model on a set of classes and using the resulting model to extend a backpack and call the Laplace library. The goal is to make predictions on an image and extract a model explanation. The documentation details the steps taken to achieve this goal.

To begin, the ResNet18 model is constructed and trained on a set of classes. I used classes_scenes.yaml which is a subclass containing images of various scenes, including outdoor landscapes and buildings. Once the model was trained, it was loaded with the weights saved to the disk. The Laplace library Github contains a code snippet that iterates through every parameter of the model, flattens each parameter, and concatenates them into one giant vector. However, we needed to use the inverse function vector_to_params() in order to get the necessary parameters to pass into the model.

The next step is to call the model on an image to make a set of predictions, with one prediction per sample. To load the tensor saved, the `torch.load` function is called. The number of vectors is obtained from index 0, and the number of samples is determined. The ResNet18 model is constructed with the number of classes in the dataset. The weights vector for each sample is obtained by looping over every sample and indexing the first dimension. The vector is converted to the correct format using the `vector_to_parameters` function, which returns a dictionary containing the parameter names as keys and the corresponding tensors as values. The parameters are loaded into the ResNet object, and the model is called on the images from the dataset. The predictions are made on each image, and the resulting output is printed to the console.

Overall, the work documentation outlines the process of training a ResNet18 model, using the Laplace library to extract intermediate layers, and making predictions on images. The process involves loading saved tensors, looping over samples to obtain weights vectors, converting the vectors to the correct format, and loading the parameters into the ResNet object. The resulting predictions are outputted for each image in the dataset.

2. Method:

The project aimed to create a tool that could provide visual explanations for Bayesian deep learning models. My work was aimed to contribute to the research Doctor Matthew Berger is doing on encoding Bayesian neural networks; documentation of his work can be found on his GitHub repository here. His work primarily obtained a last-layer laplace transformation, while my focus was particularly oriented around computing a diagonal laplace transformation.

At the beginning portion of my semester, I spent time reading various research papers related to Bayesian Neural Networks, computing Laplace transformations, and pytorch tensors, amongst similar topics. Machine learning, data visualization, and neural networks are entirely new domains to me; and throughout the semester I found that there have been more new things to learn along every single step of the process. With the help of Dr. Berger, I have had the opportunity to learn so much.

After a few changes in direction in my initial project scope, Dr. Berger and I ultimately decided on focusing my work on computing a laplace approximation along a diagonal hessian structure. However, in doing so I learned that using the original pretrained model provided by Dr. Berger was not a viable approach. Upon realizing this, I pivoted to using an older Resnet model, particularly Resnet18. This approach was favorable to the alternate which would have required writing a custom Resnet model. I then trained the Resnet18 model on the same data subset used by Doctor Berger in his work: Places365.

The Resnet18 model was fine-tuned on a set of classes specified from the Places365 dataset directory which was accessed through a symbolic link. Once the model was trained, its weights were stored in a file with the path model/resnet50-places10.pth. I then used this new model to extend backpack and call the laplace library, after adjusting to using the resnet18 model, the issue I initially encountered with the extend() function not working with the model was resolved. Rather than performing a LaPlace approximation on the last-layer, the hessian structure was specified to be diagonal. After installing the Laplace library, I computed the laplace approximation and saved the hessian matrix.

Posterior samples were generated from the Laplace approximation, and the posterior predictive distribution was evaluated to ensure the model was performing as expected. In this step, I came across another obstacle. In the laplace library github, there is a snippet of code that iterates through every parameter of the model and flattens each before concatenating them into one giant vector. This is performed using the parameters_to_vector() function. However, this vector type is not the expected object type to be passed as a parameter for running the model. Because of this, I had to use the inverse function of parameters_to_vector(), vector_to_parameters() to ensure that the parameters could be properly passed into the call statement. Once I made these adjustments, I generated samples which resulted in a vector, then I called vector to parameters and it returned the object to pass into load state dict. Doing so set the function’s parameters properly so that I could then run the model for that drawn set of parameters. Hence, after setting the parameters, the model was run for that drawn set of parameters. To make sure the proper parameters were being passed in, I ran the Laplace approximation with a break in line for testing purposes. Once the code was correct, the

Laplace approximation was run over the entire dataset, and sample() was called to return a 3D tensor called model_image_class that was stored to disk. The 3D tensor contained the probabilities for each batch (where batch size was set to 10) for each model that the sample was called on. The probability distributions were saved so that when drawing predictions, the corresponding image class file as well as model number that their probability corresponds to would be accessible.

From here, the next steps were to call the model on an image to make a set of predictions (one prediction per sample). So that I could extract an intermediate layer as a model explanation. In this script, I looped over every image, and for each image I grabbed all model predictions for each class. Then, I averaged the probabilities over the models so that I could find which class had the highest average probability. Then, I computed the uncertainty, named entropy, so that I could see how certain the predictions are and how flat the distribution is. Next, in order to get an average accuracy of the model, the number of accurate predictions divided by the total number of images was computed. Finally, I sorted the images by decreasing entropy,so that the first image in the list has the highest uncertainty. Similarly, I also sorted the class predictions so that I could return as a result the top five images with the highest uncertainty and grab the image from the directory of validation images of Places365, and compare my predictions as a human to the class predictions and entropy computed by the model so that I could determine how accurate the model’s predictions are.

3. Code Documentation:

Priolo_laplace_approx.py script:

```
import argparse

import os

import sys

import math

import torch

import torch.nn as nn

import torch.optim as optim

from torchvision.datasets import Places365

import torchvision.transforms as transforms

import torchvision.models

import yaml

from utils import *

from laplace import Laplace

from laplace.utils import ModuleNameSubnetMask

from backpack import backpack, extend

from backpack.custom_module.branching import Parallel, SumModule

from backpack.custom_module.graph_utils import BackpackTracer
```

```
from backpack.extensions import BatchGrad, DiagGGNExact

from torchvision.models import resnet18

from torch.nn.utils import vector_to_parameters #rn the utils function
from torch.nn only has the inverse function so i think i need to rewrite
that




if __name__=='__main__':

   parser = argparse.ArgumentParser()

   parser.add_argument('--net', required=True, help='name of network')

   parser.add_argument('--classes', required=True, help='yaml file for
classes')

   parser.add_argument('--data_dir', required=True, help='data directory')
#this refers to the directory of the resnet50 sample model that i have


#a symbolic link to (i.e. 'model/resnet50-places10.pth'), correct?


#NOT, the directory of where i will store the hessian matrix output,
right?


   parser.add_argument('--batchSize', type=int, default=16, help='batch
size')

   parser.add_argument('--hessian_dst', default='model/H.pth',

       help='file name of the likelihood hessian "H"') #FIXME: left off
here b4 jumping to line 76
```

```
   parser.add_argument('--cuda', dest='cuda', action='store_true',
help='enables cuda')

   parser.add_argument('--no-cuda', dest='cuda', action='store_false',
help='disables cuda')

   parser.set_defaults(cuda=True)


   opt = parser.parse_args()

   print(opt)


   device = 'cuda' if opt.cuda else 'cpu'

   is_cuda = device=='cuda'




   classes_subset = yaml.safe_load(open(opt.classes,'r'))['classes']

   train_data_loader,train_data =
places_subset_train_loader(opt.data_dir,classes_subset,opt.batchSize)

   val_data_loader,val_data,_ =
places_subset_val_loader(opt.data_dir,classes_subset,opt.batchSize)


   net = resnet18(num_classes=len(classes_subset))

   net.load_state_dict(torch.load("model/resnet50-places10.pth"))


   if is_cuda:

      net.cuda()
```

```
net.eval()

n_correct = 0

for bdx,batch in enumerate(val_data_loader):

    img_batch,class_batch = batch

    img_batch = img_batch.to(device)

    class_batch = class_batch.to(device)


    # logits

    with torch.no_grad():

        logits = net(img_batch).detach()

    #


    class_preds = torch.argmax(logits,dim=1)

    n_correct += (class_preds==class_batch).sum()

#

print('val accuracy',n_correct/len(val_data))


# use BackPACK's converter to extend the model (turned off by default)

net = extend(net, use_converter=True)




la = Laplace(
```

```
        net,

        'classification',

        subset_of_weights='all',

        hessian_structure='diag') #if this doesn't work try 'diagonal' and
then last resort try 'low_rank'

                                  #kron is referring to how we are limiting
the dependencies between networks


    print('fitting...')

    la.fit(train_data_loader)

    print('... done fitting')




    diag_samples = la.sample(n_samples=16)

    print(diag_samples.shape) #so that we can see what is being saved

    torch.save(diag_samples, os.path.join('model','posterior_samples.th'))
#where should i save this to

    #sample from laplace approx and save to disk
```

train_places.py script:

```
import argparse

import os

import sys

import math

import torch

import torch.nn as nn

import torch.optim as optim

from torchvision.datasets import Places365

import torchvision.transforms as transforms

import yaml


from torchvision.models import resnet18

from tqdm import tqdm


from utils import *


#modify this to use resnet18 model that we are using in the custom resnet
priolo py file instead of using the resnet50 that we had originally been
using


if __name__=='__main__':

   parser = argparse.ArgumentParser()

   parser.add_argument('--net', required=True, help='name of network')
```

```
   parser.add_argument('--classes', required=True, help='yaml file for
classes')

   parser.add_argument('--data_dir', required=True, help='data directory')


   parser.add_argument('--batchSize', type=int, default=64, help='batch
size')

   parser.add_argument('--lr', type=float, default=3e-3, help='learning
rate')


   parser.add_argument('--n_epochs', type=int, default=24, help='number of
epochs to take')

   parser.add_argument('--weight_decay', type=float, default=0,
help='weight_decay for SGD, also known as prior strength')

   parser.add_argument('--checkpoint_dst',
default="model/resnet50-places10.pth" , help='destinattion for model
checkpoint file')


   parser.add_argument('--cuda', dest='cuda', action='store_true',
help='enables cuda')

   parser.add_argument('--no-cuda', dest='cuda', action='store_false',
help='disables cuda')

   parser.set_defaults(cuda=True)


   opt = parser.parse_args()

   print(opt)
```

```
    device = 'cuda' if opt.cuda else 'cpu'

    is_cuda = device=='cuda'


    classes_subset = yaml.safe_load(open(opt.classes,'r'))['classes']


    init_scale = math.sqrt(1/2048)

    net = resnet18(num_classes=len(classes_subset))

    net.train()


    def set_to_eval(m):

        if isinstance(m, nn.BatchNorm2d):

            #print('batchnorm?',type(m),m.running_mean.norm())

            m.eval()

        #

    #


    train_data_loader,train_data =
places_subset_train_loader(opt.data_dir,classes_subset,opt.batchSize)

    val_data_loader,val_data,_ =
places_subset_val_loader(opt.data_dir,classes_subset,opt.batchSize)

    optimizer = optim.SGD([p for p in net.parameters() if p.requires_grad],
lr=opt.lr, momentum=0.9, weight_decay=opt.weight_decay)

    criterion = nn.CrossEntropyLoss()
```

```
if is_cuda:
    net.cuda()
    criterion.cuda()
#


for edx in range(opt.n_epochs):
    n_correct = 0
    for bdx,batch in enumerate(train_data_loader):
        img_batch,class_batch = batch
        img_batch = img_batch.to(device)
        class_batch = class_batch.to(device)


        # set grad to 0
        optimizer.zero_grad()


        # logits
        logits = net(img_batch)


        # loss
        loss = criterion(logits,class_batch)


        # backprop
        loss.backward()
```

```
        # step
        optimizer.step()

        print(edx,bdx,loss.cpu().item())

        class_preds = torch.argmax(logits,dim=1)
        n_correct += (class_preds==class_batch).sum()
    #
    print('train accuracy',edx,n_correct/len(train_data))

    alpha = 1 - (edx+1)/opt.n_epochs
    for param_group in optimizer.param_groups:
        param_group['lr'] = alpha*opt.lr
    #

    n_correct = 0
    for bdx,batch in enumerate(val_data_loader):
        img_batch,class_batch = batch
        img_batch = img_batch.to(device)
        class_batch = class_batch.to(device)

        # logits
```

```
            with torch.no_grad():
                logits = net(img_batch).detach()
            #

            class_preds = torch.argmax(logits,dim=1)
            n_correct += (class_preds==class_batch).sum()
        #
        print('val accuracy',edx,n_correct/len(val_data))
    #
    torch.save(net.state_dict(), opt.checkpoint_dst)
#
```

Vector_to_params_priolo.py script:

```
#compute and save the probabilities

import argparse

import os

import sys

import math

import torch.nn as nn

import torch.optim as optim

from torchvision.datasets import Places365

import torchvision.transforms as transforms

import torchvision.models

import yaml

from utils import *

from laplace import Laplace

from laplace.utils import ModuleNameSubnetMask

from backpack import backpack, extend

from backpack.custom_module.branching import Parallel, SumModule

from backpack.custom_module.graph_utils import BackpackTracer

from backpack.extensions import BatchGrad, DiagGGNExact


import torch

from torchvision.models import resnet18
```

```
from torch.nn.utils import vector_to_parameters #rn the utils function
from torch.nn only has the inverse function so i think i need to rewrite
that


parser = argparse.ArgumentParser()

parser.add_argument('--classes', required=True, help='yaml file for
classes')

parser.add_argument('--data_dir', required=True, help='data directory')

parser.add_argument('--cuda', dest='cuda', action='store_true',
help='enables cuda')

parser.add_argument('--no-cuda', dest='cuda', action='store_false',
help='disables cuda')

parser.add_argument('--batchSize', default = 10)

parser.set_defaults(cuda=True)

opt = parser.parse_args()

device = 'cuda' if opt.cuda else 'cpu'

is_cuda = device=='cuda'

batchSize = 10


# Load the classes_subset and data_loader

classes_subset = yaml.safe_load(open(opt.classes, 'r'))['classes']

val_data_loader, val_data, _ = places_subset_val_loader(opt.data_dir,
classes_subset, opt.batchSize)


def get_places_cats(data_dir):
```

```
    categories_file =
open(os.path.join(data_dir,'categories_places365.txt'),'r')

    cat_to_int = dict()

    cats = []

    for line in categories_file:

        line = line.rstrip()

        tokens = line.split(' ')

        cat_to_int[tokens[0]] = int(tokens[1])

        cats.append(tokens[0])

    #

    categories_file.close()

    return cats,cat_to_int


# Get the indices of the images that belong to the classes_subset

cats, _ = get_places_cats(opt.data_dir) #cats,_ =
get_places_cats(data_dir)

val_classes_file = open(os.path.join(opt.data_dir, 'places365_val.txt'),
'r') #val_classes_file =
open(os.path.join(data_dir,'places365_val.txt'),'r')

val_classes = [cats[int(line.rstrip().split(' ')[1])] for line in
val_classes_file]

data_subset = [idx for idx in range(len(val_classes)) if val_classes[idx]
in classes_subset]


#load the tensor saved
```

```
diag_samples = torch.load('model/posterior_samples.th') #know the amount
of vectors from the index 0 (the shape) is the number of samples

num_samples = diag_samples.shape[0] #Get the number of samples

net = resnet18(num_classes=len(classes_subset))

net.load_state_dict(torch.load("model/resnet50-places10.pth"))

net.cuda()

net.eval()

network = diag_samples[1]

num_images = len(data_subset)

num_classes = len(classes_subset)



# Create a tensor to store the file names for each probability

model_image_class =
torch.zeros(num_samples,num_images,num_classes).to(device)

#loop over every sample, grab the vector of parameters for each sample
(index the first dimension) to give u the weights

#then convert that vector to the correct format (vector to params
function)

#then pass that into a method of the resnet object called load state dict

class_ID = torch.zeros(num_images, dtype = torch.int32).to(device)



#Iterate over the samples

for i in range(num_samples):

   #create a counter for what model number we are on and add to index 0 of
global array
```

```
    curr_model_num = i

    # Get the vector of parameters for the current sample

    weights = diag_samples[i, :]

    # Convert the vector to the correct format using vector_to_parameters()

    # This function expects the weights vector and returns a dictionary
containing the parameter names as keys and the corresponding tensors as
values



    vector_to_parameters(weights, net.parameters())

    #net.load_state_dict(params_dict)# Load the parameters into the ResNet
object

    #now the models params are assigned and we can call the model and itll
make the predictions that we want

    #ITERATE OVER THE IMAGES from classes-scenes.yaml image dataset aND
CALL THE MODEL ON THOSE IMAGES TO SEE WHAT ITS PREDICTING

    # Now we can call the ResNet model on the images from the dataset

    # Iterate over the images and call the model on each one



    for bdx,batch in enumerate(val_data_loader):

        img_batch,class_batch = batch

        img_batch = img_batch.to(device)

        class_batch = class_batch.to(device)




        # logits
```

```
        # Compute the probabilities for the current image batch

        with torch.no_grad():

            batch_probs =
torch.nn.functional.softmax(net(img_batch).detach(), dim=1) #operates on a
batch of some number of images. for each image, returns an unnormalized
probability of a class

            #returns 2D array first image within the batch, second inddex
is indexing over classes (which is 10 for classes-scenes)

            #for each class it gives an unnormalized probability

            #want to use the one that returns the highest probability

            #applying softmax will normalize the probabilities

        if bdx == 0:

            print(i, bdx, batch_probs)

        # Compute the start and end indices for the current sample

        start_idex = bdx * batchSize

        end_idex = start_idex + batchSize

        if batchSize != class_batch.shape[0]:  # last batch

            end_idex = start_idex + class_batch.shape[0]  # adjust end
index

        # Assign the batch_probs to the corresponding slice in the second
dimension of model_image_class

        model_image_class[i, start_idex:end_idex] = batch_probs

        class_ID[start_idex:end_idex] = class_batch


#save the tensors
```

```
#saving a 3d array (model, image id, class)

torch.save(model_image_class,
os.path.join('model','model_image_class.th'))



#save the class IDs

torch.save(class_ID, os.path.join('model', 'class_ID.th'))
```

Make_predictions_priolo.py

```
import torch

#import os

#import yaml

import argparse

import os

import sys

import math

import torch.nn as nn

import torch.optim as optim

from torchvision.datasets import Places365

import torchvision.transforms as transforms

import torchvision.models

import yaml

from utils import *

from laplace import Laplace

from laplace.utils import ModuleNameSubnetMask

from backpack import backpack, extend

from backpack.custom_module.branching import Parallel, SumModule

from backpack.custom_module.graph_utils import BackpackTracer

from backpack.extensions import BatchGrad, DiagGGNExact



import torch
```

```
from torchvision.models import resnet18

parser = argparse.ArgumentParser()

parser.add_argument('--classes', required=True, help='yaml file for
classes')

parser.add_argument('--data_dir', required=True, help='data directory')

parser.add_argument('--cuda', dest='cuda', action='store_true',
help='enables cuda')

parser.add_argument('--no-cuda', dest='cuda', action='store_false',
help='disables cuda')

parser.add_argument('--batchSize', default = 10)

parser.set_defaults(cuda=True)

opt = parser.parse_args()

# Load the classes_subset and data_loader

classes_subset = yaml.safe_load(open(opt.classes, 'r'))['classes']

# define paths to files and directories

model_image_class_file = "model/model_image_class.th"

classes_file = "config/classes-scenes.yaml"

class_id_file = "model/class_ID.th"#get image id for a given index



# load model_image_class and classes

model_image_class = torch.load(model_image_class_file)

class_ID = torch.load(class_id_file) ####################



# Get the indices of the images that belong to the classes_subset
```

```
cats, _ = get_places_cats(opt.data_dir) #cats,_ =
get_places_cats(data_dir)

val_classes_file = open(os.path.join(opt.data_dir, 'places365_val.txt'),
'r')

val_classes = [cats[int(line.rstrip().split(' ')[1])] for line in
val_classes_file]

data_subset = [idx for idx in range(len(val_classes)) if val_classes[idx]
in classes_subset]


# define variables to store results

accurate_predictions = 0

class_predictions = []

entropies = []

image_IDs = []


# loop over every image

for i in range(model_image_class.shape[1]):

   # get model predictions for current image

   probs = model_image_class[:, i]

   # average over models

   bma = probs.mean(dim=0)

   # get index of class with highest averaged probability

   class_prediction_index = torch.argmax(bma)

   # get actual class index
```

```
    actual_class_index = class_ID[i]#reference the tensor i just created
###############

    # if prediction is accurate, increment counter

    if class_prediction_index == actual_class_index:

        accurate_predictions += 1

    # get class prediction

    class_prediction = class_ID[class_prediction_index]
#################################

    # compute entropy

    entropy = -(bma*torch.log(bma)).sum()

    # store results

    class_predictions.append(class_prediction)

    entropies.append(entropy)

    #store image id

    image_IDs.append(data_subset[i])#append data subset at index i
##############################


# compute average accuracy

average_accuracy = accurate_predictions / model_image_class.shape[1]


# sort images by decreasing entropy

sorted_indices = sorted(range(len(entropies)), key=lambda i: entropies[i],
reverse=True)

sorted_entropies = [entropies[i] for i in sorted_indices]

sorted_class_predictions = [class_predictions[i] for i in sorted_indices]
```

```
sorted_image_paths = [image_IDs[i] for i in sorted_indices] #sort


# print top 5 images of highest uncertainty

for i in range(5):

   print("Image path:", sorted_image_paths[i])#print

   print("Entropy:", sorted_entropies[i])

   print("Class prediction:", sorted_class_predictions[i])

   print()
```

4. Notes/Documentation on Progress:

01/06/2023-02/09/2023:

Background Research

1. Read Research Article: "*The effectiveness of feature attribution methods and its correlation with automatic evaluation scores*"
2. Read Research Article: "Explaining Bayesian Neural Networks"
3. Read Research Article: "*Averaging Weights Leads to Wider Optima and Better Generalization*"
4. Familiarize myself with PyTorch

[“The effectiveness of feature attribution methods and its correlation with automatic evaluation scores”]

Key takeaways:

- General idea: Saliency maps are known to be problematic, in that they can be unrelated to the particular class. However, simple, prototype-based explanations, can in fact outperform saliency map explanations
- Although humans are the end-users of AI model decisions, most attribution methods have only been evaluated on proxy automatic-evaluation methods, rather than by using user studies.
- After performing a user-study to see the effectiveness of attribution maps in assisting humans, it was concluded that feature attribution is not more effective than the alternative of showing humans nearest training-set examples. Rather, when showing humans fine-grained dog categorization, the attribution maps only hurt (or negatively impacted) the performance of human-AI teams compared to stand-alone AI.
- As for the future, there needs to be more conversations regarding evaluation metrics and more testing needs to be done regarding the downstream human-in-the-loop applications.
- What is an attribution map?
    - It is a heatmap that highlights the input pixels that support or deny a predicted label. I.e. the prediction is that a person has breast cancer and an attirbution map would highlight the key image pixels that supports the prediction that a human has breast or argues against it.
- The goal behind attribution maps is to allow humans to collaborate with AI decisions to better understand their decisions/conclusions. Rather than working independently,

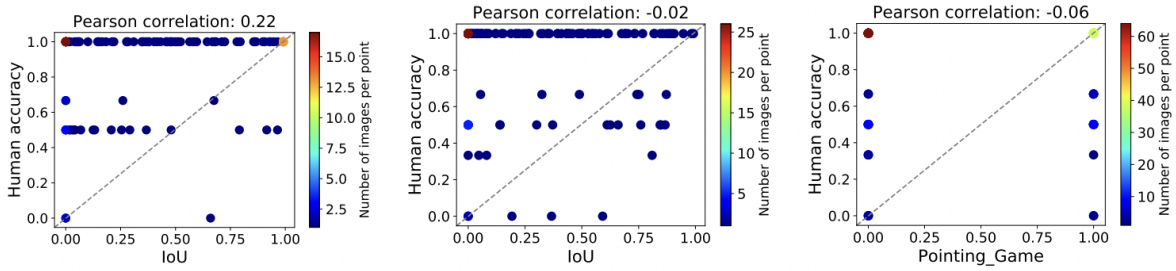
can humans work with AI to improve the accuracy of image classification, or are the two more accurate when working independently?

- Attribution maps have been found helpful in the following contexts:
    - *SOME* Image Domains:
        - Localizing tumoprs in x-rays
        - Reflecting biases
        - Memory of image classifiers
        - Teaching humans to distinguish butterflies
    - Non-image domains:
        - Deception detection
        - complementing team performance
- Visual explanations proved to hurt human accuracy in classifying fine-grained classification of dogs and it was concluded that 3NN (3 nearest neighbors images) is more useful
- 3-NN is much more useful than GradCAM (an attribution method for ImageNet classification)
- Note: What is ImageNet?
    - It’s a ~1.3M-image dataset spanning 1000 diverse categories (from man-made entities to natural entities)
- Note: What is Stanford Dogs?
    - It’s a subset of ImageNet. It’s a 120-class Stanford Dogs dataset and is much more challenging than the ImageNet tests because the dataset is smaller, the similarity between dog specifies is much higher, and users often have much less prior knowledge about the differentiation of fine-grained dog breeds than they do about the differentiation of everyday objects in ImageNet.
- Image Classifiers used: ResNet-34 pretrained on ImageNet from torchvision was used as the target classifier (the AI used) for both the imageNet and Stanford Dogs studies

- AI-only method of accepting vs rejecting a predicted label:
  - a top-1 predicted label is accepted if it's associated confidence score is ≥ T, a threshold
  - Where the optimal threshold have for ImageNet is T=0.55 and for Stanford Dogs, T=0.50
  - Idea: AI alone is determined to be more accurate than when using humans with evaluating these visual explanations (3-NN or heatmaps) so maybe the threshold for the AI is set to low, since humans often hold a higher level of confidence in order to accept a prediction (somewhere around 76% certainty). Perhaps more research into human confidence levels and decision making should be done?
- Confidence Scores only method where users see the input image, predicted label, and confidence score (no explanations are given)
  - This baseline has not been studies in computer vision and should be more closely analyzed because it determines the amount of trust a human has in an AI-decision which in turn should affect the confidence threshold that AI's should be basing decisions off of.
- GradCAM: how it works
  - White-box, gradient-based method. It relies on gradients and the activation map at the last conv layer of ResNet-34 to compute a heatmap
  - Does it act on gradients of color? Depth? Size/Spacing? Outline and slopes? What aspects are weighted more heavily for this? For example with cars model classification, outlines and shape should be weighted higher in classification accuracy rather than color (cars can come in many colors but their shape is unique), however for birds, color and slope of outline (i.e. feather colors and beak slope) are more important than size since the size of a bird can vary bird-to-bird.

- GradCAM: how it works
  - White-box, perturbation-based method. EP searches for a set of a% of input pixels that maximizes the target-class confidence score.
- Salient object detection (SOD)
  - This assess the need for heatmaps to explain a specific classifier's decisions
  - The SOD method uses PoolNet, which was trained to output a heatmap that highlights the salient object in an input image (Fig. 3)— a process that does not take our image classifier into account. Thus, SOD serves as a strong saliency baseline for GradCAM and EP.
- 3-NN
  - A prototype based explanation method
  - For a given input image x and a predicted label y, we show the top 3 nearest images to x by retrieving them from the same imageNet training-set class y
  - Does this get the closest image in the predicted label's class that match x? Why not the other way around? 3 closest images to y in the class of x?
- Can you speak more to human-annotated bounding boxes? How are they created? How is the appropriate one selected for the pointing game evaluation metric? Can there be multiple BB's used for a single image classification? i.e . a BB used for the beal of a bird, a BB used for its tail, etc?
- For Intersection over Union, what is a binarized AM? What does the method's optimal threshold "alpha" mean for

binarizing the heatmap?



(a) GradCAM's IoU (b) EP's IoU (c) EP's Pointing Game

Figure 4: The ImageNet localization performance (here, under IoU vs. human-annotated bounding boxes) of GradCAM [67] and EP [28] attribution maps poorly correlate with the human-AI team accuracy (y-axis) when users use these heatmaps in image classification. Humans often can make correct decisions despite that heatmaps poorly localize the object (see the range 0.0–0.2 on x-axis).

As GradCAM was the best attribution method in the lay-user ImageNet study (Fig. 2), we chose GradCAM as the representative for attribution maps and compare it with 3-NN on human-AI team image classification on ImageNet.

- For Weakly supervised localization (WSL) it evaluates as correct if the human-labeled BB overlaps with the binarized BB at an IoU (intersection over union) of >0.5. Why is 0.5 the selected threshold here? If BBs can only be used for ground truth labels (correctly-classified images) then what is the best metric to measure mislabeled images?

Explaining Bayesian Neural Networks

- In order to make Deep Neural Networks (DNNs) more understandable in how they make their decisions, AIworks to try and explain the interpretation and decision making process of these DNNs/ predictions. This is done through the use of heatmaps to show the patterns that lead to predictions/conclusions.
- However, Bayesian Neural Networks have a very limited form of transparency from their weight distribution, and they lack explanations of their predictions.
- Because BNNs have network weights that follow a probability distribution, their standard explanation yields an explanation distribution which thus means that by nature, BNNs use multiple prediction strategies since they follow a distribution model.
- Overall, DNNs are most useful in machine learning fields:
    - Computer vision
    - Natural language processing
    - Reinforcement learning
    - Game playing
    - Image recognition tasks
- Though the DNNs can outperform humans in many instances, explaining what features influence their predictions is difficult. (aka, they can make accurate predictions; however, it is difficult to explain to a human how these prediction are made)
- Because it is difficult to back/explain the decision making process, DNNs are largely untrusted with security critical measures (autonomous driving/personalized medicine) because the stakes are so high that transparency and understanding of decision-making in necessary for humans to accept the predictions as true/safe/accurate
- Explainable AI (XAI) is trying to address this issue by developing a way to explain the prediction making process

made by DNNs using two main methods: 1) global explanation methods and 2) local explanation methods

- Global explanation methods interpret decision making made by DNNs across a population (a prototype case) where Local explanation methods explain the process performed on a sample (particular data example)
- This is developed by training the DNNs in a maximum-a-posteriorI(MAP) setting, assigning DNN weights as point estimates. Constrastingly, BNNs learn a distribution of network weights, yielding a distribution on the prediction

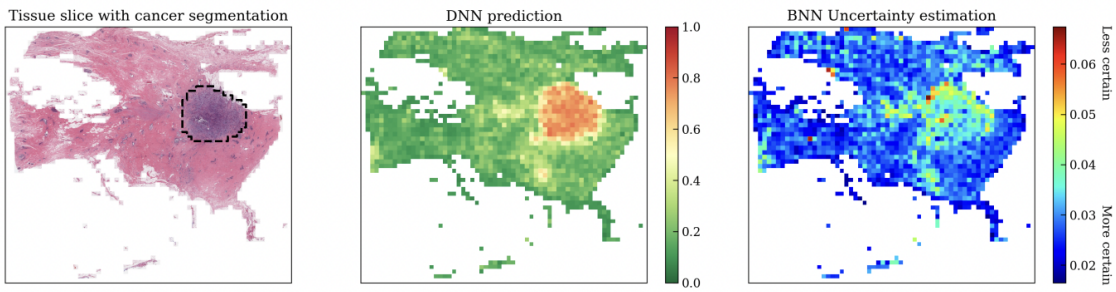


Fig. 1: Illustrating practical benefits of BNNs over regular DNNs. Left: A whole slide image from a cancer patient, divided into smaller patches. Center: The prediction accuracy for the class cancer of a standard DNN (which is identical to the mean prediction of a corresponding BNN using Laplace approximation). Right: Compared with a standard DNN, the BNN gives us additional information, in the form of uncertainties about the respective patch predictions. The Green (red) color here indicates a high (low) level of certainty.

- Mean Explanation:
    - the most simplistic option to explain the decision making process of a BNN.
- UAI: Union and Intersection Explanation
    - a practical approach that can translate the uncertainty information of a BNN's prediction into input feature uncertainty
    - This enriches the XAI explanations by providing the (un)certainty information.

- By clustering sampled explanations we can investigate the multi-modality and variability of the decision making process

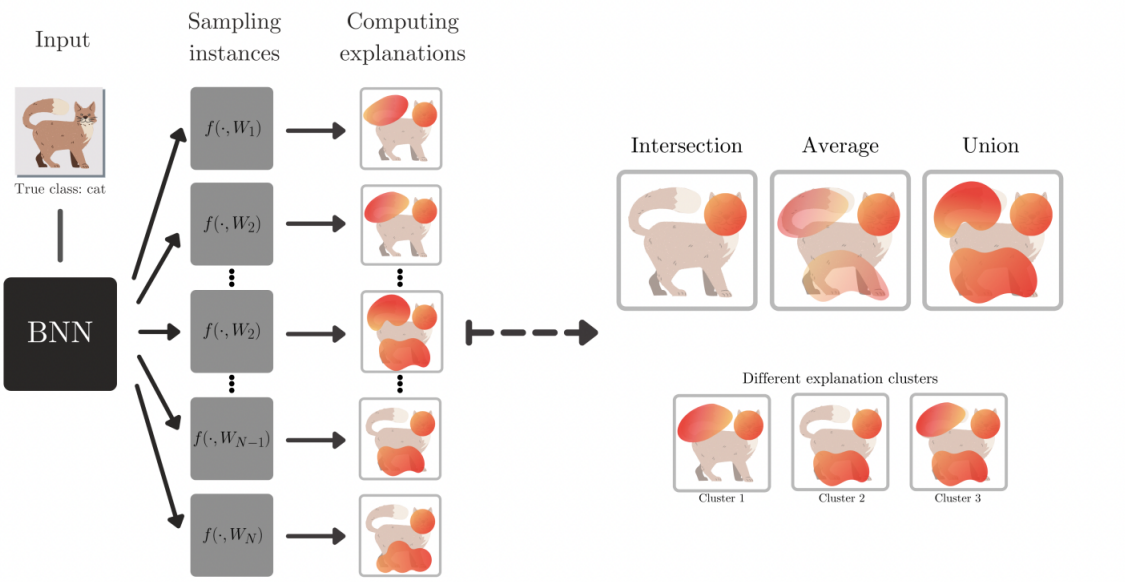


Fig. 2: Schematic illustration of proposed methods for explaining Bayesian Neural Networks. Given a particular input – a cat image – we sample models from the posterior distribution and collect local explanations. The Union explanation provides a global overview of the features learned by the BNN by combining various modes, whereas the intersection explanation provides the intersection strategy used by the BNN. Explanations can be further clustered to illustrate the main decision-making strategies.

Notes Re: PyTorch & NumPy

- What is PyTorch?
  - PyTorch is an open-source machine learning library for Python that provides a wide range of tools for deep learning and other machine learning tasks. It is primarily developed by Facebook's AI research group and is widely used in industry and academia.
- What is NumPy?
  - NumPy is a library for the Python programming language that allows for efficient manipulation of large arrays of numerical data. It is often used in conjunction with PyTorch to handle array operations.
- What are BNNs?
  - BNNs are a type of neural network that can make probabilistic predictions. They are particularly useful in situations where it is important to quantify the uncertainty of the model's predictions.
- How do PyTorch and NumPy help with BNNs models?
  - PyTorch and NumPy can be used to implement BNNs by providing the necessary tools for building and training neural networks and manipulating numerical data
- Lin Alg computations used for BNNs:
  - In a BNN, the parameters of the network are typically represented as random variables with probability distributions, rather than fixed values. Linear algebra computations are used to manipulate these distributions and compute the posterior distribution of the parameters given the data. This is a key step in the Bayesian inference process, which is used to make probabilistic predictions with BNNs.
  - Examples of linear algebra computations used in BNNs include:
    - Linear operations:

            - The neural network's linear layer is a linear operation that takes the dot product of the input with the weight matrix and add the bias.
        - Matrix Inversion:
            - Inversion of the covariance matrix of the weight is used to compute the posterior distribution of the weight.
        - Eigenvalue Decomposition:
            - used to compute the covariance matrix of the weight.
- What is a tensor?
    - Tensors are multi-dimensional arrays that are used to represent and manipulate data in machine learning and other scientific fields. They are similar to vectors and matrices, but can have more than two dimensions. Tensors can be thought of as a generalization of matrices to higher dimensions.
- Tensors' applicability to ML:
    - In machine learning and deep learning, tensors are the fundamental data structure used to represent the inputs and outputs of neural networks, as well as the parameters of the network. Tensors are used to store and manipulate large amounts of numerical data, and are often used in conjunction with linear algebra libraries such as NumPy and PyTorch.
- How do you use PyTorch tensors (matrices) and how do you work with them for computation?
    - PyTorch provides a powerful tensor library that allows for easy manipulation of matrices and other multi-dimensional arrays. Tensors in PyTorch are similar to NumPy arrays, but with the added advantage that they can be used on a GPU for faster computation.

Averaging Weights Leads to Wider Optima and Better Generalization

- Typical DNN training generalization: optimizing a loss function with an SGH variant along with a decaying learning rate until the two converge.
- Stochastic Weight Averaging (SWA) is the better DNN training generalization: averaging multiple points along the SGD trajectory with a cyclical (or constant) learning rate
- Why is a flatter solution considered better than a less flat solution?
- The weights of the networks ensembled by FGE are on the periphery of the most desirable solutions. This observation suggests it is promising to average these points in weight space, and use a network with these averaged weights, instead of forming an ensemble by averaging the outputs of networks in model space.

02/15/2023:

Goal: find a way of drawing from the posterior, different models (specify which model with a string (resnet-50) doesn't want to train a model from scratch, we want to use a pertained model

Progress documentation:

- Installed anaconda3
    - Anaconda tutorials: Anaconda Distribution Tutorial and Getting Started with Anaconda
- Downloaded DataSpell (is this the idea you recommend?)
- Installed PyTorch via anaconda
- Installed GPyTorch
- Cloned the swa_gaussian repo into my own repo
    - Go to swag -> posteriors ->swag.py is the script that trains their model:
        - (https://github.com/wjmaddox/swa_gaussian/blob/master/swag/posteriors/swag.py)
    - Go to experiments->imagenet->run_swag_imagenet.py
        - https://github.com/wjmaddox/swa_gaussian/blob/master/experiments/imagenet/eval_swag_imagenet.py
        - I want to grab imagenet, specify the model (resent-50) which will create an instance of the swag class and then run this code to gather models as it optimizes.
            - Once I find the correct model, should I add the model into the models folder, and then run only the imagenet dataset with it
            - https://github.com/timgaripov/swa/blob/master/README.md
            - On pytorch's website I found pretrained models…but do I want to get them here? If so, with or without pretrained weights? https://pytorch.org/vision/stable/models.html

02/22/2023:

Progress documentation:

- 1) the mean vector (mean of the gaussian)
- 2) the diagonal matrix (which will be a diagonal component of the matrix)
    - I want to grab imagenet, specify the model (resnet-50) which will create an instance of the swag class and then run this code to gather models as it optimizes. The model will be 1) the mean vector (mean of the gaussian) 2) the diagonal matrix (which will be a diagonal component of the matrix) 3)
    - When using swag we need to specify the # of parameters for resnet50 (it has 23 million parameters) which is the dimension of the gaussian. A gaussian takes in 2 parameters (the mean vector, and the diagonal which is represented as a vector)
    - With swag and the covariance matrix (which is 23 mil x 23 mil we want to represent as a low rank matrix)
    - Covariance matrix M= sigma + DD^(T) where sigma is the diagonal vector and DD^(T) is the low rank matrix.
    - Then the gaussian is represented by N

Steps:

- I'll need to SSH into his lab to use the GPU to run it.
- Code locally, test locally (specifying CPU), push code to github, pull from a lab computer, and run it on the lab machines gpu)
- Once I Start training and performing gradient descent on swag it'll take a while to run it
- The imagenet dataset is on his machine so he will make that accessible for me
- How do I Get my downloaded pretrained w weights resnet50 model with the run_swag_imagenet

03/01/2023:

Goal: understand how the data format for which swag returns the parameters that we will use for the second step

Progress documentation:

- Use lab computer, install python, anaconda, get the libraries that I'm prompted to get, clone and run swag can all be done locally
- going to use resnet50: can use symbolic link to access imagenet on the linux computer
- We 1st want to run the run_swag_imagenet.py script first and grab the parameters of the swag model, store it to disk, and then access it from a separate script to use to generate visual explanations from gradecam and probabilistic nearest neighbors
- This second script will be the script that we write. There's code for grad cam that we can use. The main work is taking in the swag model, formatting what swag gives us so that we can instantiate a model class (resnet50) with it
- run swag with the resnet50 model
- It is really how the data is saved and stored to disk with swag so what we need to do is identify the aspect of the model which will take some work (how do I do this?)
    -
- -once I have that I want to make predictions with the model to make sure the predictions make sense. This will check if I read in the parameters correctly and stored them into the model class correctly. Maybe I forgot to transpose the matrix, etc?

```
if args.epochs % args.save_freq != 0:
   if args.swa:
       utils.save_checkpoint(
           args.dir, args.epochs, name="swag",
state_dict=swag_model.state_dict()
       )
```

```
    else:
        utils.save_checkpoint(args.dir, args.epochs,
state_dict=model.state_dict())
```

- Then I need to sample from the Gaussian distribution (it's very simple and we need to do this ourselves)
- Goal of current stage: understand how the data format for which swag returns the parameters that we will use for the second step
- Everytime in run_swag_imagenet we create a swag class which is in swag/posteriors/swag.py
- What i'll need to know from here is the mean and squared mean values

```
def swag_parameters(module, params, no_cov_mat=True):
    for name in list(module._parameters.keys()):
        if module._parameters[name] is None:
            continue
        data = module._parameters[name].data
        module._parameters.pop(name)
        module.register_buffer("%s_mean" % name,
data.new(data.size()).zero_())
        module.register_buffer("%s_sq_mean" % name,
data.new(data.size()).zero_())

        if no_cov_mat is False:
            module.register_buffer(
                "%s_cov_mat_sqrt" % name, data.new_empty((0,
data.numel())).zero_()
            )

        params.append((module, name))
```

- These things will correspond to the parameters of every layer of the resnet50 neural network

- The cov_mat_sq is flattening the lowrank part of the gaussian approximation we want it flattened when we store it, but when we read it in we want to parse it to unflatten it to restore the structure

- The easiest way to see how to do it is by reverse engineering the swag code on how they flatten it. The SWAG class is where I Can see how they flatten it and how to access things

- As long as I create that swag class and I load in what is saved to disk, all I'll need to do is call “sample” to sample from the gaussian. This will give me a new model each time. I will need to access the parameters and leverage the swag class in order to sample the collection models.

- Call sample, it'll sample from a gaussian distribution. But we need to know how it saves and stores it to access later.
- sample_blockwise or sample_fullrank (most likely the full rank) the relevant part is self.params `for (module, name) in self.params:`
- We will need to take self.params and create an instance of our class for the neural network (vgg, resnet, etc). We will need to load in as its parameters whatever it returns in self.params
- Self.params is a collection of tensors. But nowhere is there a reference to the model itself. So we need to set the parameters of the model to elements of parameters. But how are parameters structured?
- Load in swag, call sample (that sets the instance variable parameters, which will have the sample from the gaussian), take what is stored in parameters and assign them to the parameters of the model which takes two steps A) create an instance of the model B) then set its parameters with the values from params
    - The names in swag.py will correspond to the instance variables in the model
    - I’m going to provide the model as an argument for swag

03/08/2023:

Goal: computing probabilistic nearest neighbors (what are some neighbors that are close to it whether it’s coming from a different or the same class)

    - looking at nearest neighbors under a distribution model under this posterior distribution
    - we will want to change what we apply to a laplace transformation
- At the moment, we are currently performing a laplace transformation on the very last layer of the network, but we want to change a couple lines of code to look at other layers too (which layer? We need to experiment and see)

Steps:

- Run the program from the project he is sharing with me using gradcam and using laplace instead of SWAG
- Generate a token and save it on github
- Create a symbolic link for the dataset (follow readme instructions)
- Reference the directory where the data is stored by creating a symbolic link command “ls-s-directory that exists-directory that you're creating a symbolic link for” this way I don't have to copy and paste all the data onto my local drive
- We will only use a subset of classes that are in the classes.yaml file
- I’ll need to then create the model, run the script python train_places.py --net net_name --classes classes.yaml --data_dir your_local_places_dir
- For the Compute last-layer Laplace approximation:

- We are going to want to create a new script because it’s currently only computing the laplace approximation for the last layer.
- In order to compute laplace calculations, we need to install the laplace library(did this with him)
- We want to copy and paste the laplac_approx.py script into a new one to edit it. What I’m going to want to change is around line 76:

```
la = Laplace(net, 'classification',
subset_of_weights='last_layer',
hessian_structure='full')

print('fitting...')

la.fit(train_data_loader)

print('... done fitting')

torch.save(la.H,os.path.join('model','H.th'))
```

- subset_of_weights='last_layer' instead write: subset_of_weights = ‘all’
- hessian_structure='full', instead write: hessian_structure = ‘kron’ (if that doesn't work, try diagonal and if doesnt work then try low rank)
  - What will change is how we store the results because we no longer have it simple structure
  - Note: kron is referring to how we are limiting the dependencies between networks
- Strictly speaking, we don't need to store it, but we need to use it every time (which may be expensive to compute)
- The laplace library allows you to sample models (we still only want to use the resnet-50) they have a built in sample method
  - torch.save(la.H,os.path.join('model','H.th'))

        - instead we are going to want to run la.sample but the issue here is that we are likely going to run out of GPU memory
    - In line 786 the kron class of the laplace library (https://github.com/AlexImmer/Laplace/blob/main/laplace/baselaplace.py)
    - From here it says:
        - Laplace approximation with Kronecker factored log likelihood Hessian approximation and hence posterior precision.
        - This will give me a 2d array w first dimension being sample_index and then the second dimension will be all of the parameters. Which means if we want to use it later we need to save it as a vector but as we read it in we need to set the model parameters to resnet-50
    - Near term: call sample, lookup "torch, how do I save a sample to disk" and if I don't run out of memory, next step is to figure out how can I load it to disk and set the parameters properly and then I can look into how to compute the rarest neighbors
    - Once I'm on the machine, I have sudo, so if I want to get a preferred IDE for python that works with linux then I can do that.

03/22/2023:

Updates:

We are unable to read the modules for our pretrained module resnet50 that we were using, so we need to use torch backpack extend() method instead of reading forward() from the network to the modules. There is little documentation regarding this, and what exists is likely out of date, so I Need to debug through the code, if unsuccessful, I Need to try and use the resnet18 model that is referenced in the backpack documentation on the torch website (use resnet18 trained on any dataset at first just to make sure it works with extending/reading the modules. If it works, then I Will need to train it on my own data that I Have the symbolic link for). Final approach, if all else fails, i'll need to write my own module.

Approach 1: look thru backpack documentation

1. Dig into the backpack documentation and figure out why we cannot use the nvidia resnet50 model
    a. https://docs.backpack.pt/en/master/use_cases/example_resnet_all_in_one.html#custom-resnet-with-backpack-custom-modules
2. Look into the extend() feature in the backpack documentation
    a. When you call the model with extend(what happens?
    b. Clone the backpack repository (completed)
        i. Cd backpack
        ii. Grep extend()
    c. Extend is located in __init__.py (~/backpack/backpack)-GVIM

        i. Theres a debug function that will tell u if it extended each module and we ran it in line 42 of my file and it showed that it was extending everything
    d. Thought: Could it be regarding abstract modules
3. The other option is to use the old version of pytorch's resnet18 module as referenced in the documentation instead of the new nvidia resnet module we r using
4. Maybe try this
    a. https://docs.backpack.pt/en/master/use_cases/example_resnet_all_in_one.html#any-resnet-with-backpack-s-converter

Approach 2: use an older resnet version

- First we would try resnet18 to make sure that when we call extend() does it even work?
- If so, then we will go back and train the model and then we will rerun it with extend

Approach 3: Write my own custom resnet module

- implements a resnet and uses the classes we are using

- mimicking the screenshot below from https://docs.backpack.pt/en/master/use_cases/example_resnet_all_in_one.html#any-resnet-with-backpack-s-converter:

Custom ResNet

We can build a ResNet by extending `torch.nn.Module`. As long as the layers with parameters (`torch.nn.Conv2d` and `torch.nn.Linear`) are `nn` modules, BackPACK can extend them, and this is all that is needed for first-order extensions. We can rewrite the `forward` to implement the residual connection, and `extend()` the resulting model.

> **Note**
>
> Using in-place operations is not compatible with PyTorch's `torch.nn.Module.register_full_backward_hook()`. Therefore, always use `x = x + residual` instead of `x += residual`.

```
class MyFirstResNet(Module):
    def __init__(self, C_in=1, C_hid=5, input_dim=(28, 28), output_dim=10):
        """Instantiate submodules that are used in the forward pass."""
        super().__init__()

        self.conv1 = Conv2d(C_in, C_hid, kernel_size=3, stride=1, padding=1)
        self.conv2 = Conv2d(C_hid, C_hid, kernel_size=3, stride=1, padding=1)
        self.linear1 = Linear(input_dim[0] * input_dim[1] * C_hid, output_dim)
        if C_in == C_hid:
            self.shortcut = Identity()
        else:
            self.shortcut = Conv2d(C_in, C_hid, kernel_size=1, stride=1)

    def forward(self, x):
        """Manual implementation of the forward pass."""
        residual = self.shortcut(x)
        x = self.conv2(relu(self.conv1(x)))
        x = x + residual  # don't use: x += residual
        x = x.flatten(start_dim=1)
        x = self.linear1(x)
        return x


model = extend(MyFirstResNet()).to(DEVICE)
```

04/05/2023:

- Run script:
  - <<python priolo_laplace_approx.py --net nvidia_resnet50 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
    - python priolo_laplace_approx.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365
  - <<python custom_resnet_priolo.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
  - <<python train_places.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
- Debug:
  - Key files/lines to work with in debugging:

  - Priolo_laplace_approx (Lines 46-49), Backprop_extension.py, Resnet.py, Sample_last_layer.py, Laplace_approx.py

- Look into documentation regarding the Structure from the readme at the link https://github.com/AlexImmer/Laplace/blob/main/README.md particularly regarding the implementation of the diagonal hessian structure, do I need to change the

  - 'In addition to that, subnetworks can also be specified manually, by listing the names of either the model parameters (ParamNameSubnetMask) or modules (ModuleNameSubnetMask) to perform Laplace inference over.'

- Look beyond the Structure section through the rest of the documentation and read about extendability

- Also read into this referenced article: https://arxiv.org/abs/2002.10118

    - It may be helpful as it addresses relu issues which is one of the key things we needed to change using the 'functional' import

    - Title of article: Being Bayesian, Even Just a Bit, Fixes Overconfidence in ReLU Networks

- Completed: trained the resnet18 model on the classes

- Use this new model to extend backpack and calling the laplace library (last time I couldn't do this because extend() wasn't working with the model)

- Take the model, construct the resnet18 model, and load in the weights that will be saved to the disk

04/12/2023

- In the laplace library github, there is a snippet of code that iterates through every parameter of the model and flattens each of them then concatenates it into one giant vector. We need to find this so we can do the inverse (unflatten it)
- They use a torch import function parameters_to_vector to flatten it but thank god theres the inverse import function that exists vector_to_parameters (documentation: https://pytorch.org/docs/stable/_modules/torch/nn/utils/convert_parameters.html#parameters_to_vector)
- Once this is done, call sample that gives me a vector, then call vector to parameters, this'll give me an object to pass into load state dict to set the functions parameters,
- Once we get to this point, we can run the model for that drawn set of parameters.
- In order to get to this point we need to make sure that the proper parameters are being passed in, but running the laplace approximation in its entirety isn't optimal so we need to but a break in line 383 for testing purposes of the baselaplace.py (path: home/prioloae/anaconda3/lib/python3.9/site-packages/laplace/baselaplace.py) then once we are certain the code is correct, run the laplace approximation over the entire dataset (iterate fully rather than just one iteration like we are doing right now for testing) aka, get rid of the break statement so full iteration over the whole dataset can occur rather than breaking after one iteration

1. In laplace documentation, look at the Diaglaplace class documentation (https://aleximmer.github.io/Laplace/baselaplace.html#laplace.baselaplace.DiagLaplace) and then look at the inherited members, particularly “sample”.
2. Since diaglaplace inherits baselaplace, we need to look there for inherited methods
3. In the github located at the current path: Laplace/laplace/utils/utils.py that is where the torch.nn.utils function parameters_to_vector function is imported and then used in the expand_prior_precision function at the bottom of the script. The purpose here is to Expand prior precision to match the shape of the model parameters.the return is the expanded prior precision has the same shape as model parameters
4. Currently the function is this: and since parameters_to_vector is being called,it turns the parameters into a single vector (flattens it) so theta ends up being = to 1, which triggers the first if clause to be entered but we want the elseif below it to be entered instead for the full diagonal

04/19/2023

Key Points:

- Call sample and after we run the laplace approximation, it will run sample and it'll return a vector that will store to disk which will occur several times
- Ideally we want a separate disk that'll read any of those vectors from the sample
- Have a way to draw samples

Goal: Call the model on an image to make a set of predictions (one prediction per sample). Ideally, want to be able to extract an intermediate layer as a model explanation

Terminal Commands:

Run script:

- <u>Fine-tune model on Places365 subset:</u> To fine-tune a particular model on the Places365 class subset, run the following:
  - <<python train_places.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
- <u>Compute last-layer Laplace approximation:</u> To compute the Laplace approximation, you will first need to install the Laplace library.
  - <<python priolo_laplace_approx.py --net nvidia_resnet50 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
    - Laplace approximation script: priolo_laplace_approx.py
    - Network: resnet50
    - Data directory:~/data/places-365

- <<python priolo_laplace_approx.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
    - Laplace approximation script: priolo_laplace_approx_py
    - Network: resnet18
    - Data directory:~/data/places-365
- <<python custom_resnet_priolo.py --net resnet18 --classes config/classes-scenes.yaml --data_dir ~/data/places-365>>
    - Laplace approximation script: custom_resnet_priolo.py
    - Network: resnet18
    - Data directory:~/data/places-365
    -

- Upon completion, the Hessian matrix will be stored as `model/H.th` except in my case it’s saved as model/resnet50-places10.pth.

04/26/2023:

- load the tensor saved
  - diag_samples = torch.load('model/posterior_samples.th')
- know the amount of vectors from the index 0 (the shape) is the number of samples
- loop over every sample, grab the vector of parameters for each sample (index the first dimension) to give u the weights
- then convert that vector to the correct format (vector to params function)
- then pass that into a method of the resnet object called load state dict
- now the models params are assigned and we can call the model and it'll make the predictions that we want
- Iterate over the images from classes-scenes.yaml image dataset and call the model on those images to see what it is predicting

```
import torch
from torchvision.models import resnet18
from torch.nn.utils import vector_to_parameters #rn the utils function
from torch.nn only has the inverse function so i think i need to rewrite
that

#load the tensor saved
diag_samples = torch.load('model/posterior_samples.th')
#know the amount of vectors from the index 0 (the shape) is the number of
samples
# Get the number of samples
num_samples = diag_samples.shape[0]
net = resnet18(num_classes=len(classes_subset))
#loop over every sample, grab the vector of parameters for each sample
(index the first dimension) to give u the weights
#then convert that vector to the correct format (vector to params
function)
#then pass that into a method of the resnet object called load state dict
# Iterate over the samples
for i in range(num_samples):
```

```
    # Get the vector of parameters for the current sample
    weights = diag_samples[i, :]
    # Convert the vector to the correct format using a function called
`vector_to_parameters`
    # This function should take in the weights vector and return a
dictionary containing the
    # parameter names as keys and the corresponding tensors as values
    params_dict = vector_to_parameters(weights)
    # Load the parameters into the ResNet object
    resnet.load_state_dict(params_dict)
    #now the models params are assigned and we can call the model and itll
make the predictions that we want
    #ITERATE OVER THE IMAGES from classes-scenes.yaml image dataset aND
CALL THE MODEL ON THOSE IMAGES TO SEE WHAT ITS PREDICTING
    # Now we can call the ResNet model on the images from the dataset
    # Iterate over the images and call the model on each one
    for image in dataset:
        prediction = net(image)
        print(prediction)
        # net.load_state_dict(torch.load("model/resnet50-places10.pth"))
```

05/02/2023:

Write a script that does the following:

- Loop over every image.
- For a given image, grab all model predictions. E.g. if the image was indexed by `i`, this would be: `probs = model_image_class[:,i]`. So this will return a 2D array, with the first dimension corresponding to models, and the second corresponding to classes.
- Average the probabilities over models: `bma = probs.mean(dim=0)`. So this will further reduce to a 1D array, over classes.
- Compute the index of the class that has highest averaged probability: `class_prediction_index = torch.argmax(bma)`. Further, access the image's *actual* class (specifically, the index of the class), and if these 2 indices are the same, then increment an external counter on the number of predictions that have been accurately made.
- Use this to index into the list of classes: `class_prediction = classes[class_prediction_index]`. This presumes `classes` is the list of class names we previously discussed. Store this in a list.
- Further, compute a measure of uncertainty, namely the entropy: `uncertainty = -(bma*torch.log(bma)).sum()`. The larger this is, the more uncertain we are, e.g. the flatter the distribution. Also store this in a list.
- At conclusion of the loop, divide the number of accurate predictions made by the total number of images. This gives an average accuracy. Please include this in your report.
- Sort the images by decreasing entropy, e.g. so the first image is that of highest uncertainty. Use this sorting order to also sort the class predictions.
- For, say, the top 5 images of highest uncertainty, grab the image (from the directory of validation images of Places365), its entropy, and the class prediction. You

should include these 5 instances as results in your report. The intention is that the level of uncertainty should be related to our own uncertainty about the image's class.